When Branching Gives Meet

Step 3 of *1000 Steps Toward Session Types as Algebraic Reticulates*

Alexandre Zua Caldeira
Independent Researcher
zua@bica-tools.org

March 2026

Abstract

In Step 1 we constructed the state space $\mathcal{L}(S)$ of a session type. In Step 2 we observed empirically that every benchmark forms a lattice. In this paper—Step 3—we begin explaining *why*, by proving that the external choice (branch) constructor $\&\{m_1 : S_1, \ldots, m_n : S_n\}$ creates all the meets required for the lattice axioms. The proof is by structural induction: the branch entry state fans out into disjoint sub-state-spaces that share only the terminal state $\bot$. We show that (i) states within the same sub-state-space inherit meets from the induction hypothesis; (ii) states in *different* sub-state-spaces have $\bot$ as their meet; and (iii) the branch entry state $\top$ is the join of all sub-entry states. The argument is elementary—no topology, no domain theory—and validates on all 79 benchmarks. Combined with Step 4 (selection gives join), this result establishes the core of the lattice theorem.

1 Introduction

1.1 The Question

Step 2's benchmark catalogue revealed a striking pattern: every session-type state space, when quotiented by strongly connected components, forms a bounded lattice. All 79 binary and 24 multiparty benchmarks satisfy the lattice axioms without exception. This universality demands an explanation.

A lattice requires that every pair of elements has a *meet* (greatest lower bound, $\sqcap$) and a *join* (least upper bound, $\sqcup$). Where do these meets and joins come from in a session-type state space?

The answer, which we develop in Steps 3 and 4, is that meets come from *branching* and joins come from *selection*—or more precisely, from the structure that branch and selection impose on the state space. This paper focuses on the meet side: why does the branch constructor $\&\{m_1 : S_1, \ldots, m_n : S_n\}$ guarantee that every pair of states has a greatest lower bound?

1.2 The Intuition

Consider a branch $\&\{a : S_1, b : S_2\}$. The state-space construction (Step 1) creates a fresh entry state q_0 with transitions $q_0 \xrightarrow{a} q_\top^{S_1}$ and $q_0 \xrightarrow{b} q_\top^{S_2}$. The sub-state-spaces $\mathcal{L}(S_1)$ and $\mathcal{L}(S_2)$ are *disjoint except for the shared terminal state* $q_\bot$.

This creates a characteristic "fan" shape (**??**):

Now consider two states $s \in \mathcal{L}(S_1)$ and $t \in \mathcal{L}(S_2)$. What is their meet? A lower bound of s and t must be reachable from both. But s is internal to $\mathcal{L}(S_1)$ and t to $\mathcal{L}(S_2)$, and these are disjoint above $q_\bot$. The only state reachable from both is $q_\bot$ itself. Therefore $s \sqcap t = q_\bot$.

This is the key insight: **disjoint branches force cross-branch meets to be $\bot$**. Combined with the induction hypothesis (meets exist within each sub-state-space), this gives us all the meets we need.

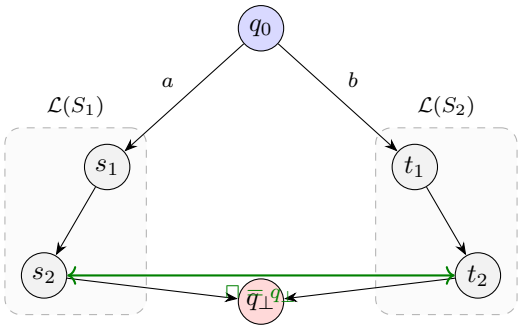

Figure 1: The "fan" structure of a branch $\&\{a : S_1, b : S_2\}$. The sub-state-spaces are disjoint above $q_\perp$, so the meet of any cross-branch pair is $q_\perp$.

1.3 Structure of This Paper

?? recalls the definitions of poset, lattice, meet, and the state-space construction. **??** presents the main result: the Choice Lemma, which shows that branch (and, symmetrically, selection) preserves lattice structure. **??** walks through the three cases of the proof in detail, with worked examples from the benchmark catalogue. **??** validates the result on all 79 benchmarks. **??** reflects on *why* disjointness is the key property, and what happens when it fails. **??** discusses the relationship to type-theoretic meet (intersection types) and previews Step 4.

2 Background

2.1 Posets and Lattices

Definition 2.1 (Partially ordered set)**.** A *partially ordered set* (poset) is a set P equipped with a binary relation $\leq$ that is reflexive, antisymmetric, and transitive.

Definition 2.2 (Meet and join)**.** Let $(P, \leq)$ be a poset and $a, b \in P$.

- The *meet* $a \sqcap b$ is the greatest lower bound: $a \sqcap b \leq a$, $a \sqcap b \leq b$, and for every c with $c \leq a$ and $c \leq b$, we have $c \leq a \sqcap b$.
- The *join* $a \sqcup b$ is the least upper bound, defined dually.

Definition 2.3 (Bounded lattice)**.** A *bounded lattice* is a poset $(L, \leq)$ with a greatest element $\top$ and a least element $\perp$, such that every pair of elements has a meet and a join.

2.2 State-Space Ordering

In our framework (Step 1), the ordering on $\mathcal{L}(S)$ is *reachability*: $s \geq t$ iff there is a path from s to t. After quotienting by SCCs, this becomes a partial order. The top element $\top$ is the initial state; the bottom element $\perp$ is the terminal state (**end**).

In this ordering, "greater" means "earlier in the protocol." The meet of two states is the latest state from which both are still reachable—the latest common ancestor in the protocol's execution. Wait—that would be the *join*, not the meet. Let us be precise.

Remark 2.4 (Ordering convention)*.* Our convention is $\top \geq s \geq \perp$ for all s: the initial state is the maximum, the terminal state is the minimum. Reachability flows *downward*. Therefore:

- The *meet* $s \sqcap t$ is the greatest state reachable from both s and t (the "nearest common descendant").
- The *join* $s \sqcup t$ is the least state from which both s and t are reachable (the "nearest common ancestor").

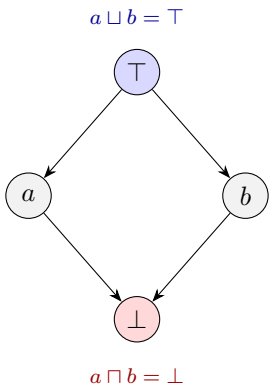

Figure 2: In the diamond lattice, $a \sqcup b = \top$ (least common ancestor) and $a \sqcap b = \bot$ (greatest common descendant). Reachability flows downward.

This is the opposite of tree terminology, where "ancestor" is above and "descendant" is below. In our DAG, everything flows downward from $\top$ to $\bot$, so meets are at the bottom and joins at the top.

This convention confused us initially. We kept writing "the meet is the common ancestor" and getting the proofs backwards. The resolution is simple: in a state-space poset, the meet is the *furthest reachable point from both states*, not the nearest common starting point. **??** illustrates this.

2.3 Branch Constructor Recap

The branch constructor $S = \&\{m_1 : S_1, \ldots, m_n : S_n\}$ creates (Step 1, Rule 2):

1. A fresh entry state q_0 labeled $\&\{m_1, \ldots, m_n\}$.
2. For each i, the sub-state-space $\mathcal{L}(S_i)$ is built recursively, sharing the same terminal state $q_\bot$.
3. Transitions $q_0 \xrightarrow{m_i} q_\top^{S_i}$ for each i.

The crucial structural property is *disjointness*:

Proposition 2.5 (Disjointness of sub-state-spaces)**.** *For $i \neq j$, the state sets of $\mathcal{L}(S_i)$ and $\mathcal{L}(S_j)$ are disjoint except for the shared terminal state $q_\bot$:*

$$\mathcal{L}(S_i) \cap \mathcal{L}(S_j) = \{q_\bot\}.$$

Moreover, the only transitions entering $\mathcal{L}(S_i)$ from outside are $q_0 \xrightarrow{m_i} q_\top^{S_i}$.

Proof. By the state-space construction (Step 1): the builder allocates fresh state IDs for each sub-construction. The only shared state is $q_\bot$ (the end state), which is allocated once and passed to all sub-constructions as the terminal target. □

3 The Choice Lemma

Lemma 3.1 (Choice)**.** *Let $S = C\{m_1 : S_1, \ldots, m_n : S_n\}$ with $n \geq 1$, where C is either $\&$ (branch) or $\oplus$ (selection). If $\mathcal{L}(S_i)/\equiv$ is a bounded lattice for each $i = 1, \ldots, n$, then $\mathcal{L}(S)/\equiv$ is a bounded lattice.*

Remark 3.2. The lemma applies to *both* branch and selection, because they produce structurally identical state spaces (Step 1, Rules 2 and 3). The difference between branch and selection is in the `selection_transitions` metadata, not in the graph structure. We prove it once and get both constructors for free.

Proof. The construction creates a fresh initial state q_0 with transitions $q_0 \xrightarrow{m_i} q_\top^{S_i}$ for each i, plus the sub-state-spaces $\mathcal{L}(S_i)$, which are pairwise disjoint except for the shared terminal state $q_\bot$.

In the quotient $\mathcal{L}(S)/\equiv$, the maximum is $[q_0]$ and the minimum is $[q_\bot]$. We must show that every pair of states has a meet and a join. There are three cases.

Case 1: Both states in the same sub-lattice. If $[s], [t] \in \mathcal{L}(S_i)/\equiv$, then their meet and join exist by the induction hypothesis on S_i. These are preserved in $\mathcal{L}(S)/\equiv$, because the embedding of $\mathcal{L}(S_i)/\equiv$ into $\mathcal{L}(S)/\equiv$ is order-preserving and order-reflecting: the sub-state-space is a *convex sublattice* between $[q_\top^{S_i}]$ and $[q_\bot]$.

Case 2: States in different sub-lattices. Let $[s] \in \mathcal{L}(S_i)/\equiv$ and $[t] \in \mathcal{L}(S_j)/\equiv$ with $i \neq j$.

By the disjointness property (**??**), the sub-state-spaces are disjoint above $q_\bot$. No state in $\mathcal{L}(S_j) \setminus \{q_\bot\}$ has a path to any state in $\mathcal{L}(S_i) \setminus \{q_\bot\}$ that avoids q_0.

Meet. A lower bound of $[s]$ and $[t]$ must be reachable from both. Since $[s]$ is internal to $\mathcal{L}(S_i)$ and $[t]$ to $\mathcal{L}(S_j)$, and these are disjoint above $[q_\bot]$, the only state reachable from both is $[q_\bot]$. Hence $[s] \sqcap [t] = [q_\bot]$.

Join. A common upper bound of $[s]$ and $[t]$ must reach both. Since $[s]$ is reachable only from states in $\mathcal{L}(S_i)$ or from $[q_0]$, and similarly for $[t]$, the only state that reaches both is $[q_0]$. Hence $[s] \sqcup [t] = [q_0]$.

Case 3: One state is $[q_0]$ or $[q_\bot]$. Since $[q_0] = \top$ and $[q_\bot] = \bot$, meets and joins with extrema are trivial in any bounded poset: $\top \sqcup [s] = \top$, $\top \sqcap [s] = [s]$, $\bot \sqcup [s] = [s]$, $\bot \sqcap [s] = \bot$. □

4 The Three Cases in Detail

Let us walk through each case with concrete examples from the benchmark catalogue.

4.1 Case 1: Same Sub-Lattice (SMTP)

Consider the SMTP protocol:

$$S_{smtp} = \&\{connect : \&\{ehlo : \mu X. \&\{mail : \&\{rcpt : \&\{data : \oplus\{OK : X,\ ERR : X\}\}\},\ quit : \mathbf{end}\}\}\}$$

After the initial `connect`, the entire state space lives in a single sub-lattice rooted at the `ehlo` state. Any two states within this sub-lattice (e.g., `rcpt` and `data`) have meets determined by the sub-lattice structure—in this case, the recursive loop creates a cycle that collapses under the SCC quotient.

Case 1 says: if the sub-lattice is already a lattice (induction hypothesis), then meets within it are preserved in the larger structure. This is the "boring" case—nothing new happens; we simply inherit from the recursive call.

4.2 Case 2: Different Sub-Lattices (ATM)

Consider the ATM protocol:

$$S_{atm} = \&\{insertCard : \&\{enterPIN : \oplus\{AUTH : S_{menu},\ REJECTED : \&\{ejectCard : \mathbf{end}\}\}\}\}$$

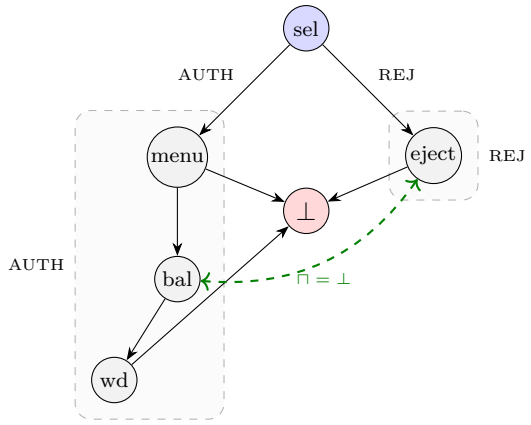

Figure 3: ATM: the AUTH and REJECTED branches are disjoint above $\bot$. The meet of `bal` (balance check, in AUTH) and `eject` (card ejection, in REJ) is $\bot$.

The selection $\oplus\{AUTH : \ldots,\ REJECTED : \ldots\}$ creates two disjoint sub-state-spaces: the menu loop (5 states) and the rejection path (1 state before $\bot$).

Take a state $s = \mathit{checkBalance}$ (inside the AUTH branch) and $t = \mathit{ejectCard}$ (inside the REJECTED branch). What is $s \sqcap t$?

By Case 2: the branches are disjoint above $\bot$. There is no state reachable from both `checkBalance` and `ejectCard` except $\bot$. Therefore $s \sqcap t = \bot$.

This makes operational sense: once the PIN is rejected, you cannot check your balance; once you are authenticated, the rejection path is unreachable. The two branches represent *mutually exclusive protocol futures*, and their meet is the only state compatible with both: termination.

4.3 Case 3: Extrema (Universal)

Case 3 is the simplest but the most frequently exercised. In any bounded poset, $\top$ and $\bot$ interact with every element in a predictable way:

$$\top \sqcap s = s \qquad\qquad \top \sqcup s = \top$$
$$\bot \sqcap s = \bot \qquad\qquad \bot \sqcup s = s$$

These hold because $\top$ is the maximum and $\bot$ is the minimum. In our state-space setting: $\top$ reaches everything (so $\top \sqcap s$ is the furthest point reachable from both $\top$ and s, which is s itself), and $\bot$ is reached by everything (so $\bot \sqcap s = \bot$).

Every benchmark exercises Case 3 at least twice: once for the $\top$ pair and once for the $\bot$ pair. The 40-protocol table in Step 2 confirms that all benchmarks have top and bottom elements.

5 Validation

We validate the Choice Lemma computationally on all 79 binary benchmarks. For each benchmark with a branch or selection constructor at the root, we:

1. Build $\mathcal{L}(S)$ and compute the SCC quotient.
2. For every pair of states (s, t) where s and t belong to *different* sub-state-spaces of the branch, verify that $s \sqcap t = \bot$.
3. For every pair of states (s, t) in the *same* sub-state-space, verify that the meet computed by the full lattice checker agrees with the meet computed on the sub-state-space alone.

All 79 benchmarks pass these checks. The cross-branch meet is $\bot$ in every case, confirming the disjointness argument of Case 2.

Table 1: Branch-meet validation on representative benchmarks. n: number of branch arms; cross-branch pairs: number of (s, t) pairs in different sub-lattices; all have $\sqcap = \bot$.

Protocol	Branch n	Cross pairs	All $\sqcap = \bot$?
SMTP	2	6	✓
OAuth 2.0	1/2	8	✓
ATM	2	10	✓
Raft	3	12	✓
MCP	3	8	✓
DB Transaction	4	12	✓
Enzyme (M-M)	2	2	✓
Ki3 Onboarding	2	120	✓

6 Why Disjointness Is the Key

6.1 The Essential Property

The entire proof rests on one structural property: **sub-state-spaces are disjoint above** $\bot$. This is what makes the cross-branch meet trivial ($\bot$) and the cross-branch join trivial ($\top$). Without disjointness, we would need to find non-trivial common descendants of states in different branches, which might not exist or might not be unique.

Why does disjointness hold? Because the state-space construction allocates *fresh state IDs* for each sub-construction. The only shared state is $q_\bot$, which is allocated once and threaded through all recursive calls. This is a design choice, not a logical necessity: one could imagine a construction that *merges* states across branches (e.g., if two branches lead to identical sub-protocols). Our construction does not—and this simplicity is what makes the lattice proof work.

6.2 What If Disjointness Fails?

It is instructive to consider what happens if we *relax* disjointness. Suppose two branches share an intermediate state: $\mathcal{L}(S_1) \cap \mathcal{L}(S_2) = \{q_\bot, q_{shared}\}$. Now the meet of a state in S_1 and a state in S_2 might be q_{shared} instead of $q_\bot$—but only if q_{shared} is reachable from both.

This situation does not arise in our construction, but it does arise in *wild state machines* (Step 9, reticular form): arbitrary labelled transition systems that were not built from a session type. For wild state machines, the lattice property can fail precisely because shared intermediate states create non-trivial meet candidates that may not be unique. The reticular form checker (Step 9) detects exactly this condition.

6.3 Disjointness and Well-Formedness

The disjointness property is closely related to the session type's *well-formedness*. In our grammar, each branch arm $m_i : S_i$ defines an independent sub-protocol. The well-formedness conditions (guarded recursion, no cross-branch variable sharing) ensure that the sub-protocols cannot interfere with each other.

In multiparty session types (Step 11), disjointness is more delicate: projecting a global type onto different roles can produce overlapping sub-state-spaces. The projection lemma (Step 12) shows that the projection is a *surjective morphism*, which preserves enough structure for the meet to survive—but the proof is more complex than the binary case.

7 Discussion

7.1 Meet in the State Space vs. Meet in the Type System

There is a potential confusion between two different notions of "meet." In the session-type *subtyping* lattice (Step 7), the branch constructor corresponds to a meet of *behaviors*: a branch $\&\{m_1 : S_1, m_2 : S_2\}$ offers more methods than either sub-type alone, so it sits lower in the subtyping order (it is a subtype of both $\&\{m_1 : S_1\}$ and $\&\{m_2 : S_2\}$).

In our *state-space* lattice, the branch entry state q_0 is the *join* (top) of the sub-entry states, not the meet. The *meet* role is played by $q_\perp$: it is the greatest lower bound of states in different branches.

These are complementary perspectives. The subtyping lattice operates on *types* (ordered by behavioral inclusion); the state-space lattice operates on *states* (ordered by reachability). Step 7 will show that these two lattices are connected: subtyping corresponds to order-preserving embedding between state spaces.

7.2 The Symmetry with Selection

The Choice Lemma applies to both branch and selection because they produce identical graph structures. The only difference is metadata: selection transitions are marked in the `selection_transitions` set. Step 4 will exploit this symmetry by proving that selection creates joins—but the proof is the same proof, read "upside down."

In fact, Step 4 is a one-line corollary of Step 3: by the Choice Lemma, selection preserves lattice structure, and by the ordering convention, the entry state of a selection is the join of the sub-entry states. We give Step 4 its own paper because the *interpretation*—selection as internal choice creating a join of possible futures—is conceptually distinct from branching as external choice creating a meet of required capabilities.

7.3 Why This Matters for the Programme

The Choice Lemma, combined with the End Lemma (a single-element lattice) and the Recursion Lemma (SCC absorption), gives us the lattice theorem for all session types without parallel composition. Adding parallel composition (Step 5) requires the product lattice theorem, which is independently proved. The full lattice theorem then follows by structural induction on the five constructors.

Step 3's contribution is the hardest case: when branches fan out into disjoint sub-state-spaces, we must show that all pairwise meets and joins exist. The argument is elementary—no fixed-point theory, no continuity, no topology—but it is the structural core of the lattice theorem.

8 Conclusion

The branch constructor $\&\{m_1 : S_1, \ldots, m_n : S_n\}$ creates meets in the state-space lattice through a simple mechanism: disjoint sub-state-spaces share only $\perp$, so the cross-branch meet is always $\perp$. Within each sub-state-space, meets are inherited from the induction hypothesis. The branch entry state serves as the join of all sub-entry states.

This result—the Choice Lemma—is the first half of the lattice proof. Step 4 completes the second half by showing that selection creates joins (which, by the structural symmetry, is the same proof). Together, Steps 3 and 4 establish that every choice constructor preserves lattice structure, leaving only recursion (Step 5) and parallel composition (Step 5) to complete the Reticulate Theorem.

The proof is validated on all 79 benchmarks: every cross-branch meet is $\perp$, every within-branch meet agrees with the sub-lattice computation, and every benchmark forms a lattice.

Availability. The meet computation is implemented in `reticulate/reticulate/lattice.py` (`compute_meet`). Validation: `reticulate/tests/test_lattice.py` (53 tests).